التوثيق الفني والتقني — Technical Documentation

معمارية وتفاصيل نظام منصة فوتيك (FuTech)

الشرح الهندسي الكامل: البنية التحتية، قواعد البيانات، خوارزمية FQS، وتكامل الـ APIs

🛠️ التقنيات المستخدمة وبنية النظام (Architecture & Tech Stack)

النظام مبني بمعمارية حديثة ومفصولة تماماً (Client-Server Architecture) عشان تضمن أعلى سرعة وأمان وسهولة في الصيانة والتطوير:

🖥️ الواجهة الأمامية (Frontend)

React 19 + TypeScript + Tailwind CSS: بتضمن سرعة رندر عالية جداً، ونظام Design System متجاوب وشغال RTL بالكامل، مع Framer Motion للحركات والتأثيرات الناعمة.

⚙️ السيرفر والباك إند (Backend)

Node.js + Express + TypeScript: بناء RESTful APIs منظمة ومكتوبة بأنواع بيانات دقيقة (Strict Types)، مع Socket.io للشات المباشر والإشعارات اللحظية.

🗄️ قاعدة البيانات (Database)

PostgreSQL + Prisma / Drizzle ORM: بنستخدم داتابيز علائقية قوية عشان نضمن سلامة البيانات والعلاقات بين المشاريع والمطورين والعروض، وإدارة الـ Migrations بسلاسة.

🐳 الحاويات والسحابة (DevOps & Docker)

Docker + GitHub Actions: توفير بيئة تشغيل معزولة (Containers) عشان المشروع يشتغل عند أي حد بضغطة زر، وخط إنتاج CI/CD لفحص الكود ونشره تلقائياً.

🗄️ شكل وتصميم جداول قاعدة البيانات (Database Schema)

قاعدة البيانات متقسمة لمجموعة جداول أساسية بتخدم كل ميزات المنصة:

اسم الجدول	أهم الحقول (Attributes)	الوظيفة باختصار
`Users`	`id, email, password_hash, role, fqs_score, hourly_rate, skills, bio`	بيانات المستخدمين (مطورين أو عملاء)، بما فيها تقييم FQS والمهارات وسعر الساعة.
`Projects`	`id, title, description, budget, deadline, stack, milestones, status, owner_id`	المشاريع المنشورة، ميزانيتها، المراحل التنفيذية، والتقنيات المطلوبة وحالتها.
`Applications`	`id, project_id, developer_id, plan, proposed_budget, proposed_duration, status`	العروض الفنية اللي بيقدمها المطورين على المشاريع مع خطة العمل المقترحة.
`Tickets`	`id, project_id, client_id, developer_id, priority, status, sla_response_hours, logs`	تذاكر الدعم الفني والصيانة بعد تسليم المشروع ومتابعة زمن الاستجابة المحدد.
`TicketReplies`	`id, ticket_id, sender_id, message, code_snippet, created_at`	الردود داخل تذكرة الدعم ومقتطفات الكود أو الرسائل المتبادلة لحل المشكلة.
`Messages`	`id, conversation_id, sender_id, body, is_read, created_at`	الرسائل الفورية للشات المباشر بين المطور والعميل أثناء سير العمل.

📊 خوارزمية مؤشر جودة المستقل (FQS Engine) بتتحسب إزاي؟

مؤشر **FQS (Freelancer Quality Score)** بيتحسب تلقائياً من 100 بعد ما المطور يخلص مشاريعه، ومش معتمد على عدد المشاريع وبس، ده بيوزن كفاءة المطور بالمعادلة دي:

درجة FQS = (معدل إكمال المشاريع × 35%) + (الالتزام بالوقت والميزانية × 30%) + (تقييمات العملاء × 20%) + (تكرار التوظيف × 15%)

شرح المعايير الأربعة ببساطة:

- **معدل الإكمال (35%):** هل المطور بيسلم المشروع كامل ولا العميل بيضطر يلغي المشروع بسبب أخطاء أو عجز عن التنفيذ؟
- **الالتزام بالوقت والميزانية (30%):** هل بيسلم في المعاد المتفق عليه بدون طلب فلوس زيادة فجأة؟
- **تقييم العملاء (20%):** متوسط النجوم (من 1 لـ 5) مضروبة في النسبة المئوية المكافئة.
- **تكرار التوظيف (15%):** هل نفس العميل رجع وظف المطور ده تاني في مشاريع تانية؟ وده أقوى دليل على احترافيته.

📦 سياسة تسليم الكود والمستودعات (Git Handover Standards)

عشان نضمن إن العميل يستلم شغل احترافي يقدر أي تيم يكمل عليه في المستقبل، المطور ملزم بالمعايير دي:

```
# شكل المستودع الإلزامي عند تسليم أي مشروع:
├── README.md # خطوات التثبيت، والتشغيل المحلي، ومتطلبات السيرفر
├── docker-
│   ├── Dockerfile # بناء الحاوية المعزولة للتطبيق
│   ├── env.example # نموذج للمفاتيح والمتغيرات بدون كتابة أسرار حقيقية
│   └── compose.yml # تشغيل الداتابيز والتطبيق بأمر واحد
├── src/ # كود المشروع مع هيكلة كلين واختبارات أساسية
└── tests/ # ملفات فحص الجودة (Unit / Integration Tests)
```

- تسليم المشروع بيكون بنقل ملكية المستودع رسمياً (Transfer Ownership) لحساب العميل على GitHub أو GitLab، مع إعطائه صلاحيات Admin كاملة.

⏱️ اتفاقية مستوى الخدمة وتذاكر الدعم (Ongoing Support & SLA)

كل مشروع بيتسلم بيكون معاه **ضمان مجاني لمدة 30 يوم** لتصليح أي عيوب أو أخطاء برمجية ظهرت في الكود، مع مستويات استجابة ملزمة:

أولوية التذكرة	الحد الأقصى للاستجابة	أمثلة على نوع المشكلة
حرج (Critical)	خلال ساعتين فقط	الموقع واقع تماماً، خلل أمني مفاجئ، السيرفر مش بيستجيب، أو الداتابيز مفصولة.
عالي (High)	خلال 6 ساعات	في ميزة أساسية مش شغالة (زي تسجيل الدخول أو إرسال الإيميلات)، وباقي الموقع شغال عادي.
متوسط / منخفض	خلال 12 - 24 ساعة	طلب تعديل شكلي بسيط، بطء في نقطة معينة، أو استفسار بخصوص التوثيق.

🔌 أهم واجهات برمجة التطبيقات (Core REST APIs)

المسار (Endpoint)	Method	الوظيفة والبيانات المرسلة
`api/auth/login/`	`POST`	تسجيل الدخول وإرجاع رمز الحماية JWT مع بيانات المستخدم.
`api/developers/`	`GET`	استرجاع قائمة المطورين مع الفلترة بالمهارات وسعر الساعة والـ FQS.
`api/projects/`	`POST`	نشر مشروع جديد بواسطة العميل مع تحديد الميزانية والمواعيد.
`api/projects/:id/apply/`	`POST`	تقديم عرض فني من المطور (خطة العمل، السعر، والمدة).
`api/tickets/`	`POST`	فتح تذكرة دعم وصيانة جديدة مع تحديد الأولوية وإرفاق اللوجز.
`api/chat/messages/`	`POST`	إرسال رسالة في الشات المباشر مع بثها فوراً عبر Socket.io للطرف التاني.